Sleep Stage Classification Using Machine Learning: A Subject-Independent Comparative Pilot Study

Atharv Upadhyaya
School of Computer Engineering
Manipal Institute of Technology
Manipal, India
[240905208] | Roll No: 30

Prithvi Warrior
School of Computer Engineering
Manipal Institute of Technology
Manipal, India
[240905230] | Roll No: 32

Hariharan VS
School of Computer Engineering
Manipal Institute of Technology
Manipal, India
[240905120] | Roll No: 23

Abstract—We compare five classical classifiers, random forest, extra trees, an RBF support vector machine, k-nearest neighbours and XGBoost, on five-class sleep staging (Wake, N1, N2, N3, REM) from a single EEG channel. The data are 13 overnight recordings from five healthy adults in a public multi-night database, cut into 13 628 labelled 30-second epochs. Each epoch is described by 22 time-domain, Hjorth-style and spectral features from channel C4-A1, and every model is evaluated on a participant it never saw in training (leave-one-subject-out). Using one epoch at a time, accuracy is 76 to 81% and macro-F1 is 0.68 to 0.72. Appending the features of the four epochs on each side lifts XGBoost to 84.9% accuracy, a macro-F1 of 0.803 and a Cohen's kappa of 0.798, and raises recall of the hardest stage, N1, from 0.38 to 0.56. Extra trees and random forest reach the highest accuracy (85.1%), while XGBoost and the SVM are the models that recover N1 best. An ablation shows that most of the gain comes from temporal context and not from using a larger boosted-tree model. The study gives a reproducible classical baseline for the database and a clear set of next experiments, including nested validation on the full 20-participant release and a deep-learning comparison on the same folds.

Index Terms—sleep staging, electroencephalography, subject-independent validation, temporal context, XGBoost, random forest, class imbalance

I. Introduction

Sleep is divided into stages that differ in brain activity, muscle tone and eye movements. In clinical practice each 30-second epoch of an overnight polysomnogram is assigned to Wake, N1, N2, N3 or rapid eye movement (REM) sleep according to the AASM manual [1], which replaced the earlier Rechtschaffen and Kales rules [2]. The resulting hypnogram is the basis for measuring sleep latency, sleep efficiency and the amount of slow-wave and REM sleep. Scoring a full night by hand takes a trained technologist a long time, and two scorers do not always agree. Inter-rater studies report overall agreement of roughly 80%, with the lowest agreement on N1 [3], [4].

That cost and variability have motivated many attempts at automatic scoring, reviewed in [5], [6]. Two families of methods dominate. In the first, a fixed set of hand-designed features is computed for each epoch and passed to a standard classifier. In the second, a neural network learns its own representation from the raw signal, often together with a recurrent layer or an attention mechanism that models the sequence of epochs [7]–[9]. Deep models currently set the highest published scores. Feature-based pipelines remain attractive when training data are scarce, when the decision has to be explained, or when the model must run on modest hardware.

Whichever family is used, reported performance depends on choices that are easy to overlook. If epochs are shuffled and split at random, epochs from the same night end up in both the training and the test set, and the classifier can exploit person-specific signal properties instead of stage-specific ones. Estimates obtained this way are optimistic for a new patient [10]. Holding out entire participants gives a more honest estimate. A second choice is how much of the neighbourhood of an epoch the classifier is allowed to see. Stages persist for minutes, and the scoring rules themselves refer to the epochs before and after the one being scored, so a model that sees a single 30-second window discards information that a human scorer uses.

This paper studies both choices on a small public dataset [11]. We extract 22 features from one EEG channel, train five classical classifiers, and evaluate them with leave-one-subject-out (LOSO) cross-validation, first on one epoch at a time and then with the features of neighbouring epochs appended. The contributions are:

1) a documented pipeline from raw EDF files to a 22-feature representation of each 30-second epoch, together with the processed feature matrix;
2) a context-aware representation that adds the features of up to four neighbouring epochs on each side within a recording, and an ablation that separates the effect of context from the effect of model size;
3) a LOSO comparison of five classifiers with overall, per-stage and per-participant results, so that the weak N1 stage and the spread between people are visible and not averaged away.

The study is a pilot on five participants and one EEG channel, and it provides a reproducible classical baseline for this database. Section VII sets out the experiments that build on it.

II. Related Work

A. Feature-based sleep staging

Classical staging systems extract a vector of descriptors from each epoch and train a conventional classifier. The

descriptors are usually drawn from three groups: amplitude statistics, Hjorth's time-domain parameters of activity, mobility and complexity [12], and power in the standard EEG frequency bands. Memar and Faradji built a multi-class system on features from a single EEG channel and a random forest [13]. Fraiwan et al. combined time-frequency features of one channel with a random forest [14], and Lajnef et al. used a decision-tree arrangement of multi-class support vector machines [15]. Reviews of the area point out that N1 is the stage that automatic methods find hardest [5], [6]. The present study follows the same recipe and adds a participant-level evaluation, a per-stage analysis and a systematic test of temporal context.

B. Deep and sequence models

Deep models take the raw signal or a time-frequency image as input. DeepSleepNet combines convolutional feature learning on single-channel EEG with a bidirectional recurrent layer over the epoch sequence [7]. SeqSleepNet treats sleep staging as a sequence-to-sequence problem and labels a run of epochs at once [8], and AttnSleep adds an attention mechanism to single-channel convolutional features [9]. The common thread is that the label of an epoch is predicted from a window of surrounding epochs. This is the idea behind our neighbouring-epoch features, applied to hand-designed descriptors and not to learned ones.

C. Evaluation and class imbalance

Published scores are hard to compare because datasets, montages, label sources and splits differ [6]. Subject-wise splits avoid leakage between training and test sets and approximate the use case of scoring a person the model has not seen [10]. Sleep data are also imbalanced, with N2 several times more frequent than N1. Learning from imbalanced data is a topic of its own [16], and a classifier can reach a high accuracy while ignoring the minority class. We therefore use class weights in training and report macro-averaged metrics and per-class recall next to accuracy.

III. Dataset and Labels

A. Participants and recordings

The recordings come from the continuous multi-night sleep database of Ying et al. [11]. The full release contains 20 healthy adults recorded on three consecutive nights with five EEG channels, two EOG channels and one chin EMG channel, sampled at 500 Hz, and sleep stages scored by three physicians from different sleep centres. We work with the project subset covering participants SLP001 to SLP005 and not with the whole database. Our scripts found 14 EDF files in the subset. Thirteen produced labelled epochs: three nights each for SLP001, SLP002 and SLP003, and two nights each for SLP004 and SLP005. SLP004 has no usable second night, and the remaining file contributed no epochs.

The 13 recordings give 13 628 labelled epochs (Table I). The epochs per participant range from 1 807 (SLP004) to 3 395 (SLP003). N2 is the largest class with 5 547 epochs (40.7%), while N1 has 999 (7.3%). The N1 count varies a great deal between people, from 99 epochs for SLP005 to 345 for SLP003, which matters when N1 is held out.

TABLE I
Labelled epochs per participant and stage.

Participant	Total	Wake	N1	N2	N3	REM
SLP001	3 214	312	200	1 486	513	703
SLP002	3 063	626	213	1 264	252	708
SLP003	3 395	484	345	1 337	610	619
SLP004	1 807	338	142	789	275	263
SLP005	2 149	712	99	671	314	353
All	13 628	2 472	999	5 547	1 964	2 646

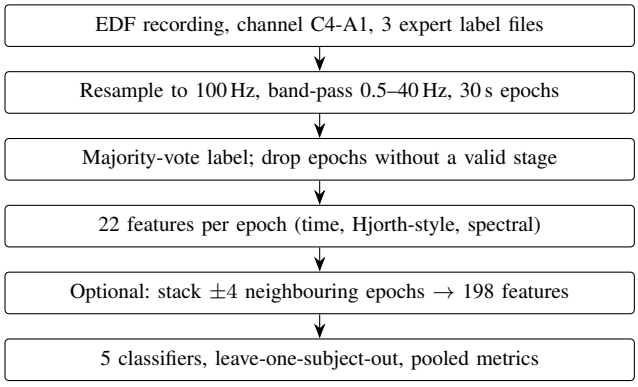


Fig. 1. Processing pipeline from raw recording to pooled held-out metrics.

B. Reference labels

Every recording comes with up to three expert label files, each holding one stage code per 30-second epoch. The target of an epoch is the most frequent code across the available files. Codes were read as 0 for Wake, 1 for N1, 2 for N2, 3 for N3 and 4 for REM, and the class proportions in Table I are consistent with that reading. Epochs whose code falls outside this set were removed. When all three scorers disagree, the implementation returns the code of the first file in sorted order, so a tie is settled by one fixed scorer and not by any rule about the stages. We did not count how often ties occur. A majority vote also discards the information that an epoch was contested, which means that the scores reported below measure agreement with a consensus label and not with any single clinician.

IV. Methods

A. Pipeline overview

Fig. 1 summarises the processing. A recording is read, reduced to one channel, resampled and band-pass filtered, and cut into epochs. Each epoch becomes a feature vector. For the context-aware experiment the vectors of neighbouring epochs are stacked. Classifiers are then trained and tested with one participant held out at a time.

TABLE II
FEATURE GROUPS COMPUTED FOR EACH 30-SECOND EPOCH.

Group	Count	Contents
Amplitude statistics	7	mean, std, min, max, RMS, skewness, kurtosis
Hjorth-style	4	σ_0, σ_1/σ_0, σ_2/σ_1, and a variance ratio
Band power	5	delta, theta, alpha, sigma, beta (absolute)
Relative band power	5	band power / total power
Spectral entropy	1	entropy of the normalised spectrum
Total	22	

B. Signal preprocessing

Signals were read with MNE-Python [17]. The code picks C4-A1 and falls back to F4-A1, O2-A1 or P4-A1 if it is missing, but all 13 recordings contained C4-A1. The channel was resampled from 500 to 100 Hz and filtered with a fourth-order Butterworth infinite-impulse-response band-pass between 0.5 and 40 Hz. The filtered signal was cut into non-overlapping epochs of 30 s (3 000 samples), and samples after the last complete epoch were discarded. No artefact rejection was applied, so the features and classifiers are tested on the signal as recorded, which keeps the pipeline simple and fully reproducible. EOG and EMG were left for later work: chin EMG tone and eye movements are the cues that separate REM from Wake and N1, so adding them is a natural extension.

C. Single-epoch features

Each epoch x is centred on its mean and described by 22 numbers (Table II).

For the Hjorth-style group let v_0, v_1 and v_2 be the variances of the epoch and of its first and second differences, and $\sigma_i = \sqrt{v_i}$. The four values are σ_0, the mobility $\sqrt{v_1/v_0}$, the mobility of the first difference $\sqrt{v_2/v_1}$, and $\sqrt{\max(0,\, v_0 v_2 - v_1^2)}/v_1$. These follow Hjorth's activity, mobility and complexity [12] but are not identical to them, so we call them Hjorth-style. The first value repeats the standard deviation from the amplitude group, and we kept it for simplicity.

The power spectral density is estimated with Welch's method [18] using 4-second segments (400 samples) with the SciPy defaults of a Hann window and 50% overlap. Band power is the trapezoidal integral of the density over delta (0.5–4 Hz), theta (4–8 Hz), alpha (8–12 Hz), sigma (12–16 Hz) and beta (16–30 Hz). Relative power divides by the integral over 0.5–40 Hz. Spectral entropy [19] is computed on the density normalised to sum to one over 0.5–40 Hz,

$$H = -\sum_k p_k \ln p_k, \qquad p_k = \frac{P(f_k)}{\sum_j P(f_j)}, \tag{1}$$

with the natural logarithm and no division by the number of bins. Low values indicate power concentrated in a few frequencies, as in slow-wave sleep, and high values a flatter spectrum, as in wakefulness. Non-finite values, if any, were replaced by zero.

D. Context features

A single 30-second window is a poor basis for deciding between stages that blend into each other. For the context-aware experiment the five absolute band powers were first transformed as $\log(x + 10^{-12})$, because they span several orders of magnitude. For every epoch we then concatenated the 22-feature vector of the epoch with the vectors at offsets $-4, \ldots, -1$ and $+1, \ldots, +4$ in the same recording, giving $22 \times 9 = 198$ features. At the start or end of a recording the nearest available epoch is repeated. No label of a neighbour is used, only its signal features, so a held-out participant contributes no label information at test time.

Two properties of this construction are worth stating. Offsets count retained labelled epochs, so if an unlabelled epoch was removed earlier, the neighbours on either side of the gap become adjacent. And the construction uses epochs after the current one, which a real-time system cannot do.

E. Classifiers

Five classifiers were trained using scikit-learn [20] and the XGBoost library, with the seed fixed to 42.

- Random forest [21]: 180 trees in the single-epoch experiment and 300 with context, minimum leaf size 2, balanced-subsample class weights.
- Extra trees [22]: the same sizes, minimum leaf size 2, balanced class weights. Split thresholds are drawn at random, which makes training faster and often smooths the decision surface.
- RBF support vector machine [23]: $C = 3$, balanced class weights, inputs standardised with training-fold statistics.
- k-nearest neighbours: $k = 9$, distance weighting, standardised inputs, no class weights.
- XGBoost [24]: 25 trees of depth 3 with learning rate 0.1 in the single-epoch experiment; 300 trees of depth 4 with learning rate 0.05, row subsampling 0.8 and column subsampling 0.7 in the context experiment. Balanced sample weights are computed from each training fold.

Hyperparameters were set by hand and not searched over, so that differences between models reflect the algorithms and not tuning effort. The single-epoch XGBoost was kept deliberately small as a lightweight baseline.

F. Evaluation protocol and metrics

In each of the five LOSO folds all epochs of one participant form the test set and the other four participants form the training set. Predictions from the five folds are pooled to compute overall metrics, and fold-level accuracy is also reported. The pooled score is epoch-weighted, so participants

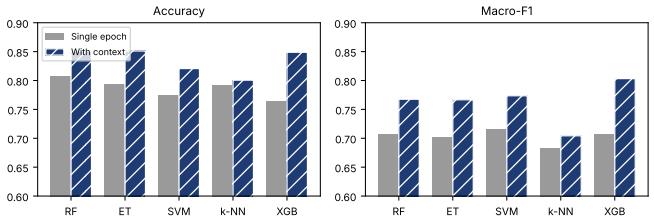


Fig. 2. Pooled LOSO accuracy and macro-F1 for each classifier with and without neighbouring-epoch features.

with more epochs count for more. Accuracy is the fraction of epochs labelled correctly. For class c,

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{2}$$

$$F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{3}$$

Macro-F1 is the unweighted mean of the five $F1_c$ and weighted-F1 weights them by class size. Cohen's kappa [25] measures agreement beyond what the label frequencies alone would produce. Macro-F1 is the main metric for comparing models because N2 makes up 41% of the epochs and accuracy can be high while N1 is mostly missed.

V. Results

A. Single-epoch features

Table III (left block) gives the pooled results when each epoch is classified from its own 22 features. Random forest has the best accuracy (0.807) and the SVM the best macro-F1 (0.716). XGBoost, with its 25 shallow trees, has the lowest accuracy (0.764) but a macro-F1 (0.707) close to the others. k-NN has the lowest macro-F1 (0.682). Differences between models are small compared with the differences between participants: for XGBoost the accuracy on the held-out participant ranges from 0.721 (SLP003) to 0.812 (SLP005).

B. Effect of temporal context

With context (right block of Table III) four of the five models gain between 4 and 9 points of accuracy. XGBoost gains the most, 8.5 points of accuracy and 9.7 points of macro-F1, and becomes the best model by macro-F1 (0.803) and kappa (0.798). Extra trees and random forest tie on accuracy at 0.851, with extra trees ahead in the fourth decimal. k-NN gains less than one point of accuracy. A likely reason is that distances in 198 dimensions are dominated by many correlated, partly irrelevant features, but we did not test this. Fig. 2 shows the two settings side by side.

C. Context or model size?

The XGBoost configuration differs between the two experiments, so part of its gain could come from using 300 trees of depth 4 instead of 25 trees of depth 3. Table IV keeps the larger configuration fixed and changes only the input. On a single epoch the larger model reaches 0.785 accuracy and 0.725 macro-F1. Model size therefore accounts for about two points of accuracy, and context for the other six: a window of ± 2 epochs gives 0.845 and ± 4 gives 0.849. A window of ± 8 is slightly better again (0.854, macro-F1 0.810), but we kept ± 4 as the main setting because it was chosen before the wider window was tried. Gains flatten quickly with window size, which suggests that most of the useful context lies within a few minutes of the current epoch. Standardising each feature within its recording lowered accuracy in both settings. It removes real differences in stage composition between nights, which is one explanation, but we did not test it.

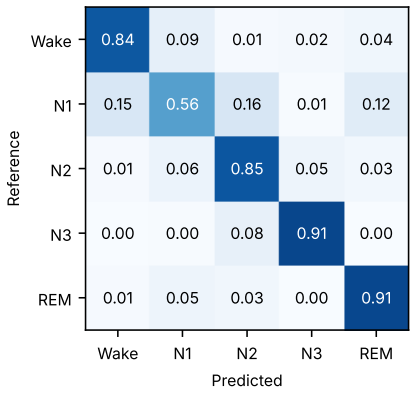


Fig. 3. Row-normalised confusion matrix of XGBoost with context, pooled over the five held-out participants.

D. Per-stage behaviour

Table V lists precision and recall for XGBoost per stage. Context improves every stage, and N1 most of all: recall rises from 0.38 to 0.56 and precision from 0.23 to 0.45. Even so, N1 has an F1 of 0.50 against 0.87 or 0.88 for each of the other four stages. Fig. 3 shows where the N1 errors go. Of the 999 N1 epochs, 562 are classified correctly, 161 are labelled N2, 153 Wake and 116 REM. In the other direction, 327 N2 epochs are labelled N1, the largest single source of false N1 labels. The confusion of N1 with Wake and REM is expected from one EEG channel without EOG or EMG, since slow eye movements and muscle tone are among the cues scorers use there.

The models differ in how they trade precision against recall on N1 (Table VI). With context, random forest and extra trees label N1 conservatively: precision is 0.62 and 0.68, but recall is only 0.22 and 0.21. The SVM and XGBoost label it more freely and recall 0.54 and 0.56 of N1 epochs at the cost of precision. The overall accuracy of the two groups is within a few points, so accuracy alone hides this difference. Which behaviour is preferable depends on the use. A screening tool might accept false N1 labels to avoid missing transitions, whereas a tool that reports N1 duration would prefer fewer false positives. We did not adjust decision thresholds, which would move each model along its own precision–recall curve.

TABLE III
POOLED LEAVE-ONE-SUBJECT-OUT RESULTS ON 13 628 EPOCHS. "SINGLE EPOCH" USES THE 22 FEATURES OF THE EPOCH ALONE; "WITH CONTEXT" APPENDS THE FEATURES OF FOUR EPOCHS ON EACH SIDE (198 FEATURES). BEST VALUE PER COLUMN IN BOLD.

	Single epoch			With context		
Model	Acc.	Macro-F1	κ	Acc.	Macro-F1	κ
Random forest	**0.807**	0.707	**0.736**	0.851	0.768	0.794
Extra trees	0.794	0.702	0.722	**0.851**	0.767	0.795
SVM (RBF)	0.774	**0.716**	0.703	0.821	0.774	0.761
k-NN	0.792	0.682	0.716	0.801	0.704	0.725
XGBoost	0.764	0.707	0.689	0.849	**0.803**	**0.798**

TABLE IV
XGBOOST (300 TREES, DEPTH 4) WITH DIFFERENT INPUTS. "Z-SCORED" MEANS EACH FEATURE IS STANDARDISED WITHIN ITS OWN RECORDING.

Input	Feat.	Acc.	Macro-F1	κ
Single epoch	22	0.785	0.725	0.715
Single, z-scored	22	0.748	0.687	0.666
Context ± 2	110	0.845	0.798	0.793
Context ± 4	198	0.849	0.803	0.798
Context ± 8	374	0.854	0.810	0.804
Context ± 4, z-scored	198	0.832	0.787	0.775

TABLE V
PER-STAGE PRECISION (P), RECALL (R) AND F1 FOR XGBOOST, SINGLE EPOCH (25 TREES) AND WITH CONTEXT.

	Single epoch			With context		
Stage	P	R	F1	P	R	F1
Wake	0.81	0.76	0.78	0.90	0.84	0.87
N1	0.23	0.38	0.28	0.45	0.56	0.50
N2	0.93	0.76	0.84	0.92	0.85	0.88
N3	0.79	0.91	0.84	0.85	0.91	0.88
REM	0.77	0.81	0.79	0.86	0.91	0.88

TABLE VI
PER-CLASS RECALL AND N1 PRECISION WITH CONTEXT. R: RECALL, P: PRECISION.

Model	Wake R	N1 R	N2 R	N3 R	REM R	N1 P
Random forest	0.89	0.22	0.91	0.86	0.92	0.62
Extra trees	0.87	0.21	0.91	0.87	0.94	0.68
SVM (RBF)	0.78	0.54	0.81	0.93	0.90	0.40
k-NN	0.70	0.15	0.87	0.83	0.96	0.50
XGBoost	0.84	0.56	0.85	0.91	0.91	0.45

TABLE VII
ACCURACY ON EACH HELD-OUT PARTICIPANT WITH CONTEXT.

Model	SLP001	SLP002	SLP003	SLP004	SLP005
Random forest	0.846	0.846	0.827	0.848	0.905
Extra trees	0.839	0.841	0.840	0.847	0.904
SVM (RBF)	0.807	0.846	0.820	0.847	0.783
k-NN	0.809	0.755	0.799	0.806	0.852
XGBoost	0.848	0.860	0.817	0.851	0.883

E. Per-participant results

Table VII gives the accuracy on each held-out participant. The spread across participants within one model is six to ten points. SLP005 is the easiest participant for random forest, extra trees, XGBoost and k-NN and the hardest for the SVM (0.783), and SLP003 is the hardest for random forest and XGBoost. XGBoost has the highest accuracy on SLP001, SLP002 and SLP004, extra trees on SLP003 and random forest on SLP005, by margins between 0.1 and 1.4 points. Because each fold holds a single participant, differences between models on one participant cannot be separated from ordinary person-to-person variation, and five folds are too few to support a statistical test of the model ranking.

VI. DISCUSSION

A. What the experiments show

The clearest result is that temporal context is worth more than the choice of classifier. At fixed model size, adding eight neighbouring epochs raised XGBoost by six points of accuracy, more than the largest gap between any two classifiers in the single-epoch comparison (four points). This is what one would expect from the way stages are scored, and it matches the design of sequence models [7], [8]. It also changes the task. The classifier no longer labels an epoch from that epoch alone, and a device that has to report the current stage immediately cannot use the four future epochs. A causal variant that uses only past epochs would need to be evaluated separately.

The second result is that the ranking of models depends on the metric. Extra trees and random forest lead on accuracy, XGBoost leads on macro-F1, weighted-F1 and kappa, and the SVM comes close to XGBoost on macro-F1 while trailing it on accuracy. All of this follows from N1. Where an application needs reliable N1 detection, macro-F1 and N1 recall are better guides than accuracy, and for that purpose XGBoost or the SVM would be chosen over the tree ensembles on this evidence.

B. Practical implications

For a researcher who needs a fast, inspectable staging baseline, the results suggest a concrete recipe: compute a few dozen spectral and statistical features per epoch, stack the features of the neighbouring epochs, and train a boosted-tree model. This takes minutes on a laptop, needs no GPU, and reaches 85% accuracy and a macro-F1 of 0.80 on participants the model has not seen. The feature importance of such a model can

be read directly, which is harder with a neural network. For applications that must report N1 reliably, XGBoost or the SVM are the better choices, and for applications that care mainly about overall agreement the tree ensembles are equally good and simpler.

VII. Future Work

This pilot provides a reproducible classical baseline that can be extended in several directions. Running the same pipeline on all 20 participants of the database would allow confidence intervals, significance tests between models, and nested selection of the context window and hyperparameters. Adding the EOG and EMG channels should improve the separation of Wake, N1 and REM, and a past-only context window would show how much of the gain carries over to real-time use. The three expert annotations can be used to train on soft labels or to measure agreement with each scorer, and threshold tuning or a dedicated N1 detector can raise N1 performance further. Finally, a CNN and a CNN–LSTM trained on the same epochs, folds and labels would complete the comparison between classical and deep models, and a feature-importance analysis could show which of the 22 descriptors a low-power device needs to keep.

VIII. Conclusion

On 13 recordings from five healthy adults, classical classifiers working from 22 features of one EEG channel reach 76 to 81% accuracy on unseen participants when each epoch is classified alone. Adding the features of four neighbouring epochs on each side raises accuracy to 85% for random forest, extra trees and XGBoost, and XGBoost reaches a macro-F1 of 0.803 and recalls 56% of N1 epochs. An ablation attributes about six of the eight points gained by XGBoost to context and about two to the larger model. N1 remains the weakest stage for every classifier. The pipeline is fully reproducible, runs in minutes on a laptop, and provides a baseline against which larger cohorts, additional sensors and deep models can be measured. Scaling to the full 20-participant database with nested validation, adding the EOG and EMG channels, and running a deep-learning baseline on the same folds are the immediate next steps.

Reproducibility and Data Availability

The code, the processed 22-feature matrix, epoch metadata, fold metrics, class-wise reports and figures are included in the project archive. The original EDF recordings are not redistributed. They should be obtained from the dataset's official OSF record (doi: 10.17605/OSF.IO/GX36M) under its terms. The scripts `build_dataset.py`, `train_classical.py`, `train_context.py`, `ablation.py` and `make_figures.py` regenerate every table and figure in this paper from the feature matrix, and the scripts use a fixed seed. Small differences in the last digit can appear across library versions.

Acknowledgment

The authors thank Manipal Institute of Technology, Manipal Academy of Higher Education, for the academic environment and institutional support for this student project, and the creators of the continuous multi-night sleep database for releasing the recordings and expert annotations. No external funding was used.

References

[1] R. B. Berry et al., The AASM Manual for the Scoring of Sleep and Associated Events: Rules, Terminology and Technical Specifications, version 2.6. Darien, IL, USA: American Academy of Sleep Medicine, 2020.

[2] A. Rechtschaffen and A. Kales, Eds., A Manual of Standardized Terminology, Techniques and Scoring System for Sleep Stages of Human Subjects. Los Angeles, CA, USA: UCLA Brain Information Service/Brain Research Institute, 1968.

[3] H. Danker-Hopfe et al., "Interrater reliability for sleep scoring according to the Rechtschaffen & Kales and the new AASM standard," J. Sleep Res., vol. 18, no. 1, pp. 74–84, 2009, doi: 10.1111/j.1365-2869.2008.00700.x.

[4] R. S. Rosenberg and S. Van Hout, "The American Academy of Sleep Medicine inter-scorer reliability program: Sleep stage scoring," J. Clin. Sleep Med., vol. 9, no. 1, pp. 81–87, 2013, doi: 10.5664/jcsm.2350.

[5] R. Boostani, F. Karimzadeh, and M. Nami, "A comparative review on sleep stage classification methods in patients and healthy individuals," Comput. Methods Programs Biomed., vol. 140, pp. 77–91, 2017, doi: 10.1016/j.cmpb.2016.12.004.

[6] L. Fiorillo et al., "Automated sleep scoring: A review of the latest approaches," Sleep Med. Rev., vol. 48, Art. no. 101204, 2019, doi: 10.1016/j.smrv.2019.07.007.

[7] A. Supratak, H. Dong, C. Wu, and Y. Guo, "DeepSleepNet: A model for automatic sleep stage scoring based on raw single-channel EEG," IEEE Trans. Neural Syst. Rehabil. Eng., vol. 25, no. 11, pp. 1998–2008, 2017, doi: 10.1109/TNSRE.2017.2721116.

[8] H. Phan, F. Andreotti, N. Cooray, O. Y. Chén, and M. De Vos, "SeqSleepNet: End-to-end hierarchical recurrent neural network for sequence-to-sequence automatic sleep staging," IEEE Trans. Neural Syst. Rehabil. Eng., vol. 27, no. 3, pp. 400–410, 2019, doi: 10.1109/TNSRE.2019.2896659.

[9] E. Eldele et al., "An attention-based deep learning approach for sleep stage classification with single-channel EEG," IEEE Trans. Neural Syst. Rehabil. Eng., vol. 29, pp. 809–818, 2021, doi: 10.1109/TNSRE.2021.3076234.

[10] S. Saeb, L. Lonini, A. Jayaraman, D. C. Mohr, and K. P. Kording, "The need to approximate the use-case in clinical machine learning," GigaScience, vol. 6, no. 5, Art. no. gix019, 2017, doi: 10.1093/gigascience/gix019.

[11] S. Ying, J. Fu, L. Wang, H. Zhang, Y. Yu, H. Wang, J. Xie, and T. Liu, "A continuous multi-night sleep database for exploring sleep structure similarity," Sci. Data, vol. 12, Art. no. 946, Jun. 2025, doi: 10.1038/s41597-025-05202-6.

[12] B. Hjorth, "EEG analysis based on time domain properties," Electroencephalogr. Clin. Neurophysiol., vol. 29, no. 3, pp. 306–310, 1970, doi: 10.1016/0013-4694(70)90143-4.

[13] P. Memar and F. Faradji, "A novel multi-class EEG-based sleep stage classification system," IEEE Trans. Neural Syst. Rehabil. Eng., vol. 26, no. 1, pp. 84–95, 2018, doi: 10.1109/TNSRE.2017.2776149.

[14] L. Fraiwan, K. Lweesy, N. Khasawneh, H. Wenz, and H. Dickhaus, "Automated sleep stage identification system based on time–frequency analysis of a single EEG channel and random forest classifier," Comput. Methods Programs Biomed., vol. 108, no. 1, pp. 10–19, 2012, doi: 10.1016/j.cmpb.2011.11.005.

[15] T. Lajnef et al., "Learning machines and sleeping brains: Automatic sleep stage classification using decision-tree multi-class support vector machines," J. Neurosci. Methods, vol. 250, pp. 94–105, 2015, doi: 10.1016/j.jneumeth.2015.01.022.

[16] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.

[17] A. Gramfort et al., "MEG and EEG data analysis with MNE-Python," Front. Neurosci., vol. 7, Art. no. 267, 2013, doi: 10.3389/fnins.2013.00267.
[18] P. D. Welch, "The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms," IEEE Trans. Audio Electroacoust., vol. 15, no. 2, pp. 70–73, Jun. 1967, doi: 10.1109/TAU.1967.1161901.
[19] T. Inouye et al., "Quantification of EEG irregularity by use of the entropy of the power spectrum," Electroencephalogr. Clin. Neurophysiol., vol. 79, no. 3, pp. 204–210, 1991, doi: 10.1016/0013-4694(91)90138-T.
[20] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
[21] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
[22] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," Mach. Learn., vol. 63, no. 1, pp. 3–42, 2006, doi: 10.1007/s10994-006-6226-1.
[23] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273–297, 1995, doi: 10.1007/BF00994018.
[24] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
[25] J. Cohen, "A coefficient of agreement for nominal scales," Educ. Psychol. Meas., vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.